

Applied 2: Presentation Script

755 spoken words. Target: 6:20, with 40 seconds available before seven minutes. At an assumed 130 words per minute, speech takes about 5.8 minutes, leaving time for pauses and slide changes. This is a planning estimate, not a measured rehearsal.

Suggested split: Savin presents slides 1–6; Shamle presents slides 7–11. Stop at the conclusion; the appendix is not part of the script.

Savin Vindiv De Alwis (35221631) — sdea0018@student.monash.edu

Shamle Thilaksiri (35512075) — wthi0003@student.monash.edu

Italic cues are directions; speak only the main paragraphs.

Slide 1 — Introduction

Savin • 10 seconds • Finish by 0:10

Start on the cover.

We're Savin and Shamle. We'll cover GPU image rotation, then the main challenges of scaling HPC for AI.

Slide 2 — Moving the image to the GPU

Savin • 30 seconds • Finish by 0:40

Follow the arrows from storage to the final host buffer.

First, the CPU loads and decodes the image into host RAM and allocates GPU buffers. DMA then transfers the pixels over PCI Express. The GPU rotates them, and we copy the finished result back for saving. Pinned RAM helps transfers; pageable RAM may need staging. GPU memory bandwidth and transfer bandwidth are different, so both affect performance.

Slide 3 — How the rotation works

Savin • 35 seconds • Finish by 1:15

Point to the inverse-mapping equations, then the output-to-input diagram. Do not read the equations aloud.

For rotation, each thread handles one output pixel. It works backwards using this inverse rotation to find the nearest input pixel, then copies its RGB values. Outside the image, we write black. Each thread writes somewhere different, so there's no need for atomics or a barrier. With our y-down coordinates, positive angles are counterclockwise. The trade-off is possible cropping and jagged edges from nearest-neighbour sampling.

Task 1: CUDA and GPUDirect Storage

Slide 4 — CUDA threads, blocks and grids

Savin • 40 seconds • Finish by 1:55

Point to the launch, the hierarchy and the short brightness example. Do not read every code line.

The CPU launches a grid of blocks, and each block contains threads. Here, sixteen by sixteen gives two hundred and fifty-six threads per block, or eight warps. For the image size shown, that's sixty-four by sixty-four blocks. Each thread finds its pixel and checks the boundary. Streaming multiprocessors, or SMs, schedule resident warps using SIMT, with thirty-two threads per warp. This helps hide delays, but registers and branching can limit it. The second snippet adds brightness and clamps the result.

Slide 5 — Expected speedup

Savin • 30 seconds • Finish by 2:25

Emphasize the full workload, rather than only the kernel.

The GPU can process lots of pixels at once, but memory access matters. Coalesced writes help; rotated reads can be less efficient. We'd compare the same rotation against single-threaded and multithreaded CPUs, counting setup, transfers and completed GPU work. Streams can overlap work when hardware and pinned buffers allow it. Serial work still limits speedup. We haven't measured a speedup yet.

Slide 6 — Where GPUDirect Storage helps

Savin • 30 seconds • Finish by 2:55

Compare the two storage paths. Hand over after the final sentence.

GPUDirect Storage can skip host staging and transfer storage data straight into GPU memory. The CPU still controls I/O through cuFile. It doesn't decode images or speed up the rotation kernel. Our CPU-decoded pipeline still needs host RAM. We'd consider GDS for supported, storage-bound batches with GPU-ready data, then measure the benefit. Fallback may still use host staging. Shamle will cover responsible HPC scaling.

Task 2: responsible HPC scaling

Slide 7 — Energy and environmental costs

Shamle • 55 seconds • Finish by 3:50

Point to 485 and 950 on the first chart. Briefly indicate the overlapping categories on the second.

Adding GPUs gives us more capacity, but it also adds communication and resource costs. Finishing faster doesn't always mean using less energy. The IEA's 2026 report estimates data-centre electricity use at four hundred and eighty-five terawatt-hours in 2025, and projects nine hundred and fifty for 2030. That covers the whole sector, not just AI training. The growth categories on the right overlap, so we can't add them. Green AI focuses on efficiency per useful result, while water research shows that location and timing matter too. We'd track whole-job energy, carbon and cooling impacts, including failed runs and tuning. Manufacturing the hardware also has an environmental cost.

Slide 8 — Security, governance and fairness

Shamle • 55 seconds • Finish by 4:45

Explain the security side, then move to the fairness side.

Distributed training means gradients and checkpoints move between workers, so there are more things to protect. Zhu's study shows that exposed gradients can sometimes reveal training inputs. That doesn't mean every system leaks. We'd use restricted access, isolation and protected transfers and storage, with clear rules for data permissions, copies and deletion. Dual-use risks also need an accountable owner. Fairness is a separate issue. Gallegos's survey shows why the tests, datasets and mitigations matter. More GPUs don't automatically make a model fair. We'd reserve compute to test affected groups, check their results separately, and test again after making changes. Encryption protects data; it doesn't guarantee fair outputs.

Task 2: responsible HPC scaling

Slide 9 — Who gets access to compute?

Shamle • 35 seconds • Finish by 5:20

Keep the emphasis on participation, allocation and the trade-off.

Access to compute affects who can train models and reproduce research. Ahmed and Wahed look at historical institutional participation, while Strubell's study looks at workload costs. Their scopes differ, but both highlight barriers. NAIRR is a US response; it doesn't prove worldwide equality. We'd publish GPU-hour allocation rules, support smaller institutions, and offer training and appeals. Reserved capacity can reduce utilization or delay urgent work, so waiting times need monitoring too.

Slide 10 — Our responsible-use framework

Shamle • 40 seconds • Finish by 6:00

Follow Map, Measure and Manage. Indicate that Govern applies throughout.

Our proposal brings together OECD principles, NIST risk management, Green AI and carbon-aware scheduling. Govern means someone is accountable throughout. Map means understanding the use, the people affected and the alternatives. Measure means checking quality, energy, security and group harms. Manage means deciding whether to approve, defer, reduce scale or redesign. We'd shift flexible jobs to lower-carbon periods only if deadlines, water limits and fair waiting times still allow it. Urgent work and uncertain forecasts can justify exceptions, but we'd record and review those decisions.

Slide 11 — Conclusion

Shamle • 20 seconds • Finish by 6:20

Finish here. Do not continue into the appendix.

So, CUDA suits independent pixel work, but we still need to verify the output and measure performance. Responsible HPC also needs protected data, fair access and useful results. Next, we'd compile and check the rotation, then measure a real cluster workload.